

Assignment-1

1. Explain the major functions of an Operating System with examples.

An operating system (OS) is a software that manages computer hardware and software resources and provides common services for computer programs. The primary functions of an operating system include:

1. Process Management

- **Scheduling:** The OS decides which process to execute next and allocates CPU time to each process.
- **Process Synchronization:** The OS ensures that multiple processes can access shared resources without conflicts.
- **Deadlock Handling:** The OS prevents and resolves deadlocks, which occur when two or more processes are waiting for each other to release resources.

2. Memory Management

- **Memory Allocation:** The OS allocates memory to processes and manages the allocation and deallocation of memory space.
- **Virtual Memory:** The OS uses virtual memory to allow processes to access more memory than is physically available.
- **Caching:** The OS caches frequently accessed data in memory to improve performance.

3. File Management

- **File Organization:** The OS organizes files into a hierarchical file system, making it easy to find and access files.
- **File Access Control:** The OS controls access to files and ensures that only authorized users can access specific files.
- **Backup and Recovery:** The OS provides mechanisms for backing up files and recovering them in case of system failure.

4. Device Management

- **Device Drivers:** The OS provides device drivers that allow hardware devices to communicate with the computer.
- **Device Scheduling:** The OS schedules access to devices and ensures that devices are used efficiently.
- **Error Handling:** The OS handles device errors and ensures that the system continues to function properly.

5. Security

- **User Authentication:** The OS provides mechanisms for user authentication, such as passwords and biometric data.
- **Access Control:** The OS controls access to system resources and ensures that only authorized users can access specific resources.
- **Encryption:** The OS can encrypt data to protect it from unauthorized access.

6. Networking

- **Network Communication:** The OS provides mechanisms for computers to communicate over a network.
- **Internet Access:** The OS provides support for accessing the Internet and using network services.
- **Network Security:** The OS provides mechanisms for securing network communications and protecting the system from network attacks.

7. User Interface

- **Command-Line Interface (CLI):** The OS provides a CLI that allows users to interact with the system using text commands.
- **Graphical User Interface (GUI):** The OS provides a GUI that allows users to interact with the system using graphical elements such as icons, menus, and windows.

8. System Utilities

- **Text Editors:** The OS provides text editors that allow users to create and edit text files.
- **File Managers:** The OS provides file managers that allow users to manage files and folders.
- **System Monitoring Tools:** The OS provides tools for monitoring system performance and identifying potential problems.

These are just some of the essential functions of an operating system. The specific features and capabilities of an OS may vary depending on its design and purpose.

Additional Functions of Operating System

1. File Management

- The operating system provides an interface for users to create, delete, and modify files.
- It also manages the storage of files on secondary storage devices, such as hard disks and optical drives.
- The operating system keeps track of the location of files on secondary storage devices and provides mechanisms for users to access files.

2. Memory Management

- The operating system manages the allocation and deallocation of memory to processes.
- It keeps track of which areas of memory are in use and which areas are free.
- The operating system also manages the swapping of processes between main memory and secondary storage.

3. Process Management

- The operating system creates and terminates processes.
- It also schedules processes to run on the CPU.
- The operating system manages the resources that are allocated to each process, such as memory, CPU time, and I/O devices.

4. Device Management

- The operating system manages the devices that are connected to the computer.
- It provides an interface for users to access devices and it also handles the interrupts that are generated by devices.
- The operating system also manages the allocation and deallocation of devices to processes.

5. Networking

- The operating system provides support for networking.
- It allows processes to communicate with each other over a network.
- The operating system also manages the routing of network traffic.

6. Security

- The operating system provides security features to protect the computer from unauthorized access.
- It also provides mechanisms for users to authenticate themselves to the operating system.
- The operating system also manages the access control to files and other resources.

7. User Interface

- The operating system provides a user interface that allows users to interact with the computer.
- The user interface can be a command-line interface (CLI) or a graphical user interface (GUI).
- The operating system also provides mechanisms for users to customize the user interface.

2. Compare Monolithic, Layered, and Microkernel OS Structures.

Comparison of OS Structures

Feature	Monolithic	Layered	Microkernel
Structure	All OS services run in kernel space as a single process.	Organized as a hierarchy of layers; each layer uses the one below it.	Core services run in kernel; other services run in user space.
Performance	High: Fast communication due to shared address space.	Moderate: Can be slow due to traversing multiple layers.	Low: Slower due to increased inter-process communication (IPC).
Reliability	Low: One component failure can crash the entire system.	Moderate: Easier to debug, but layers depend on lower levels.	High: Failure of one service doesn't crash the entire system.
Flexibility	Low: Hard to modify or add new features.	Moderate: Easier to maintain than monolithic.	High: Highly modular and easy to extend.
Examples	Linux, MS-DOS.	THE Operating System, Windows NT (partially).	Minix, QNX, Mach.

3. What is a Virtual Machine? Explain its architecture.

A Virtual Machine (VM) is a software-based computer that runs within a physical computer (known as the host). It emulates the behavior of a physical machine and can run its own operating system (guest OS) and applications independently just like a real computer.

For example, you can run a Linux VM inside a Windows PC using tools like VirtualBox or VMware.

Working of Virtual Machines

A VM uses virtualization technology to abstract the hardware of a physical machine (CPU, memory, storage, etc.) into a virtual environment. This is managed by a special software called a hypervisor, which allocates system resources to multiple VMs.

Each VM behaves like a separate computer, even though it's running on shared hardware. It includes:

- Virtual CPU
- Virtual memory
- Virtual disk (storage)
- Network interface card (NIC)

Example: When you run multiple processes on a regular OS, they appear to have separate CPUs and memory (thanks to CPU scheduling and virtual memory). A VM extends this illusion to the OS level itself.

Virtual Machines Diagram

Types of Virtual Machines

There are two main types of VMs:

1. Process Virtual Machine

- Designed to run a single application (or process).
- Example: Java Virtual Machine (JVM), which allows Java programs to run on any platform.

2. System Virtual Machine

- Emulates an entire physical machine.
- Can run a full OS like Linux, Windows, or macOS.

- Example: VMs created using VirtualBox, VMware, or Hyper-V.

How to Set Up a Virtual Machine

Setting up a VM involves the following steps (example for installing a Linux VM):

1. Create a new virtual machine
2. Allocate virtual disk space
3. Attach a virtual network adapter
4. Install the guest operating system (Linux)
5. Register and activate the VM (if needed)
6. Install necessary drivers or guest additions

Most platforms like VirtualBox or VMware offer wizards to simplify this setup.

Uses of Virtual Machines

- Running multiple OSs on a single machine
- Testing new software or OS safely (sandboxed)
- Running legacy applications
- Server virtualization (hosting multiple VMs on a single server)
- Secure development and debugging (isolate risky tasks)

Hypervisor

A hypervisor is software (sometimes firmware or hardware-based) that enables virtualization by separating physical resources from VMs.

There are two types:

- Type 1 (Bare Metal): Runs directly on hardware (e.g., VMware ESXi, Microsoft Hyper-V)
- Type 2 (Hosted): Runs on top of a host OS (e.g., VirtualBox, VMware Workstation)

The hypervisor is responsible for assigning CPU, memory, disk, and network to each VM.

Advantages of Virtual Machines

- Isolation: Each VM is fully independent and secure from others.
- Flexibility: Run different OSs on one device.
- Cost & energy saving: Multiple VMs can run on a single machine.
- Disaster recovery: Easy to back up, clone, and restore.
- Legacy support: Run old apps on modern hardware.
- Simplified testing: Perfect for software development and security testing.

Types of Virtualization (Beyond VM)

Virtualization is not limited to just virtual machines. Here are 5 major types:

1. Hardware Virtualization (Server Virtualization)

Hardware virtualization is the process of creating multiple virtual machines (VMs) that run on a single physical server using a hypervisor.

- A hypervisor (like VMware ESXi or Microsoft Hyper-V) directly interacts with the physical hardware.
- It allocates physical resources (CPU, memory, storage) to each VM.
- Each VM operates independently with its own OS and applications.

Benefits:

- Efficient hardware utilization.
- Cost savings (fewer physical servers needed).
- Scalability and easier deployment of services.
- Fault isolation: if one VM crashes, others remain unaffected.

2. Software Virtualization

Software virtualization allows an operating system or application to run in an environment that is different from its native platform.

- A software layer mimics the required hardware or operating system.
- Example: Running Android OS on a Windows PC using an emulator like BlueStacks.

Benefits:

- Cross-platform compatibility.
- Testing apps on different OS versions.

- Reduced software conflicts

3. Storage Virtualization

Storage virtualization combines multiple physical storage devices into a single logical storage pool, making management easier and more flexible.

- Software abstracts the physical storage from multiple devices.
- The system presents the unified storage as a single resource to users and applications.

Benefits:

- Centralized storage management.
- Better performance through load balancing.
- Easier scaling and disaster recovery.

4. Network Virtualization

Network virtualization abstracts network resources, allowing multiple virtual networks to be created on a single physical infrastructure.

- Combines hardware (switches, routers) and software resources into a virtual network.
- Divides bandwidth into independent channels for better resource allocation.

Benefits:

- Improved network efficiency and security.
- Easier management and scaling.
- Dynamic provisioning of network resources.

5. Desktop Virtualization

Desktop virtualization separates the desktop environment from the physical machine and stores it on a remote server.

- Users access their virtual desktop over a network using any device (PC, tablet, thin client).
- All processing and data remain on the central server.

Benefits:

- Centralized control and security.

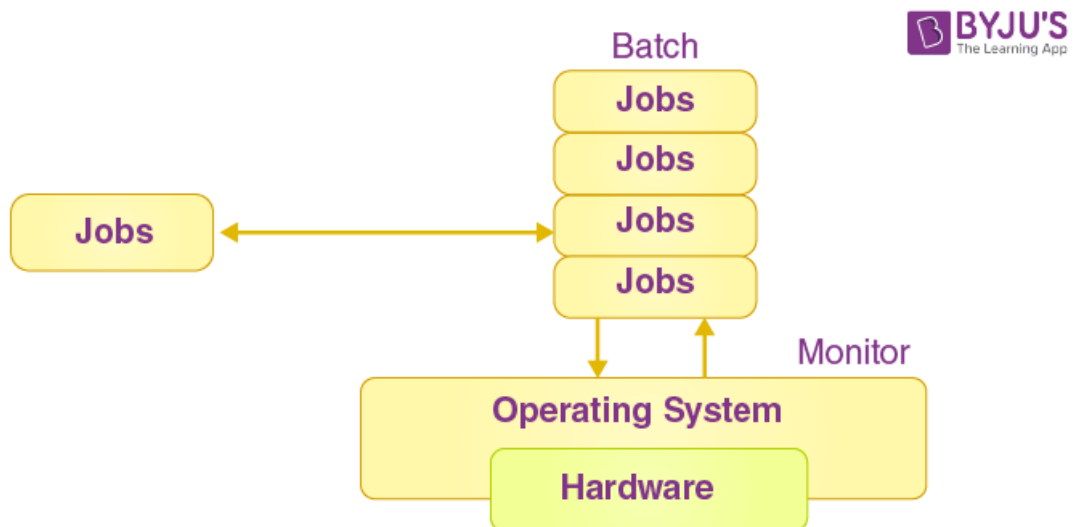
- Easier software updates and patches.
- Flexibility for remote work.

4. Explain different types of OS (Batch, Time-sharing, Real-time).

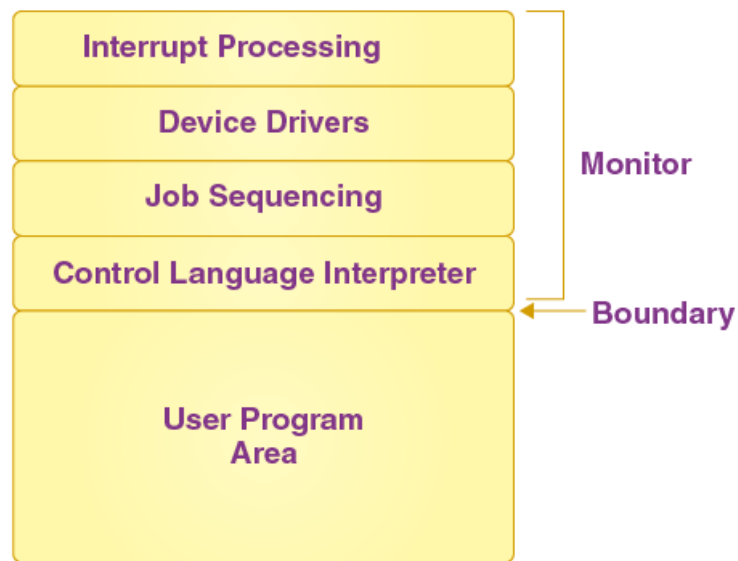
1. Batch Operating System

Batch processing was quite popular in the 1970s. Similar types of jobs have been batched together and completed on time using this method. People were very much accustomed to utilising a single computer, known as a mainframe.

Now, more than one individual can have access to this Operating System, and they can submit their jobs to the system for execution. The system places all of the jobs in a queue, first come, first served, and then executes them one at a time. When all of the jobs are completed, the users receive their respective outputs.



The fundamental goal of this Operating System was to transfer the control from one of the jobs to another as soon as it was finished. It had a small group of programs known as the resident monitor. These were always contained in one section of the main memory. The remaining portion is used for maintenance work. Learn more about [Batch Operating System](#) here.



Memory Layout of the Resident Monitor

Pros of Batch OS

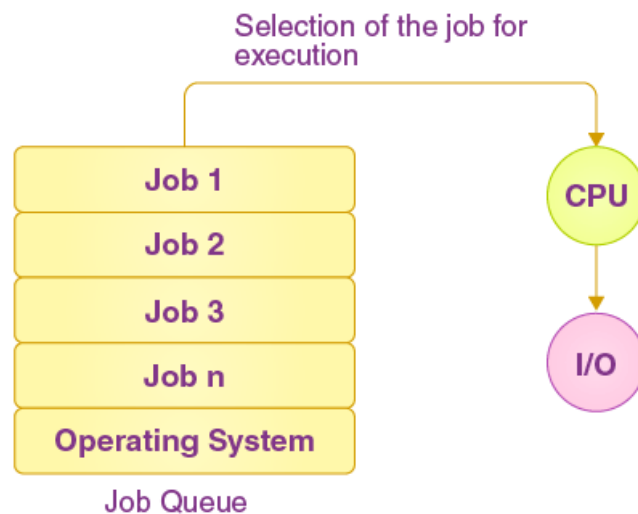
The usage of a resident monitor helps in increasing the computer efficiency by reducing the time the CPU spent switching between two tasks.

Cons of Batch OS

Not Interactive: Batch Processing is not appropriate for jobs that rely on user input. Because the user is not present at the time of execution, a job that demands the input of two numbers from the console will never obtain it in the batch processing scenario.

Starvation: The batch processing suffers from a lack of resources.

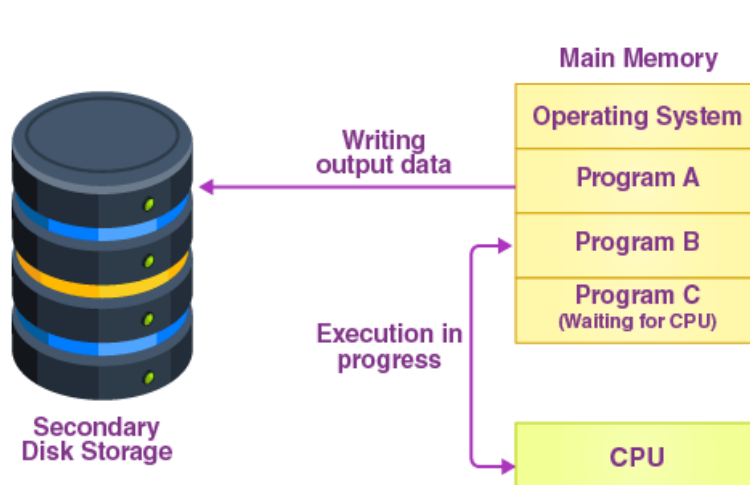
Consider the following scenario:



In the batch, there are five jobs and they are J1, J2, J3, J4, and J5. In case J1 takes a considerably long time to execute, the other four tasks will either never be executed or will have to wait a long time. As a result, the other processes suffer.

2. Multiprogramming Operating System

Multiprogramming is a variation of batch processing in which the CPU is kept busy at all times. CPU time and IO time are two forms of system time required by each process. When a process completes its I/O in a multiprogramming environment, the CPU can begin the execution of other processes. As a result, multiprogramming helps in improving the system's efficiency. Read more on [Multiprogramming Operating System](#) here.



Jobs in multiprogramming systems

Pros of Multiprogramming OS

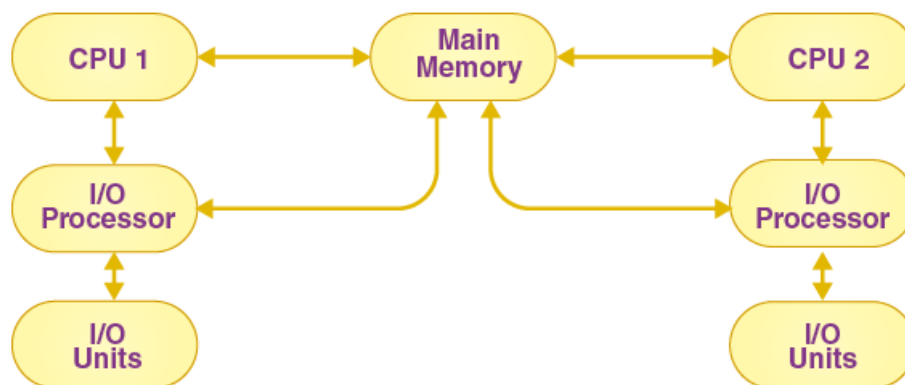
It increases the job throughput of the system since the CPU continually runs one program. It is also possible to shorten response times since the resources are used pretty smartly.

Cons of Multiprogramming OS

Multiprogramming systems create an environment in which multiple system resources are efficiently utilised, but they do not allow for any user interaction with the computer.

3. Multiprocessing Operating System

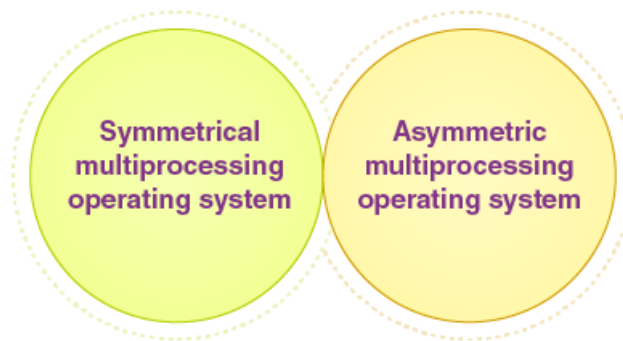
Multiprocessing helps in performing parallel computing. There are several processors in a system, each of which can run multiple processes at the same time. The system's throughput will be significantly increased as a result of this.



Working of Multiprocessor System

Parallel computing is performed by multiprocessing. The presence of more than one processor in the system allows it to run multiple processes at the same time, increasing the system's throughput. Read more on [Multiprocessing Operating System](#) here.

Types of Multiprocessing System



Pros of Multiprocessing OS

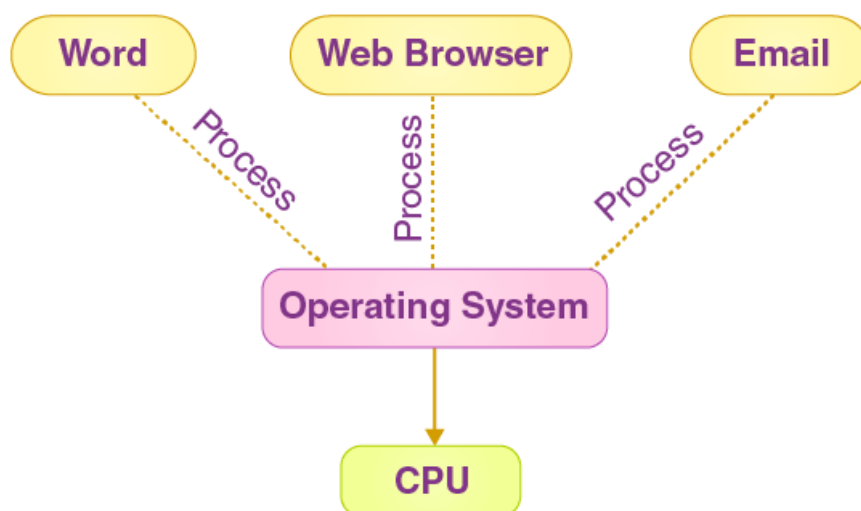
Increased reliability: Processing duties can be spread among numerous processors in the multiprocessing system. This improves reliability because if one processor happens to fail, the work can be passed on to another to finish.

Increased throughput: More jobs can be performed in less time as the number of processors increases.

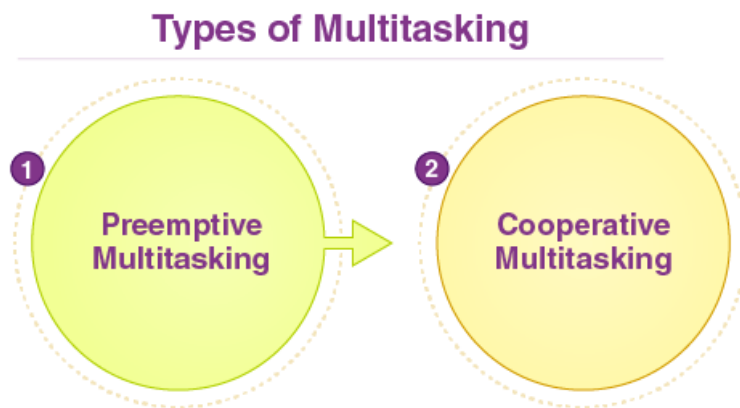
Cons of Multiprocessing OS

Because it manages multiple CPUs at the same time, a multiprocessing OS is more complex and advanced.

4. Multitasking Operating System



The multitasking OS refers to a logical extension of the multiprogramming Operating System, which allows users to run many programs at the same time. It enables a user to complete multiple computer tasks at the same time. Read more on the [Multitasking operating System](#) here.



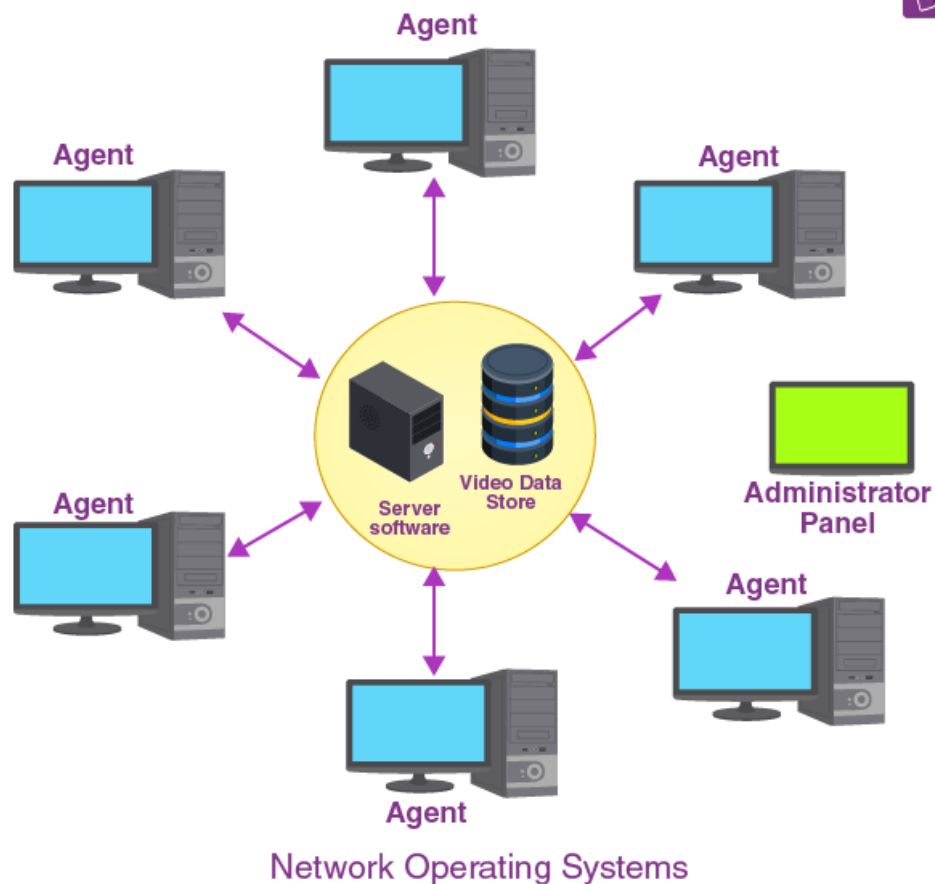
Pros of Multitasking OS

This Operating System is better adapted to handling several users at the same time. Memory management is well-defined in multitasking Operating Systems.

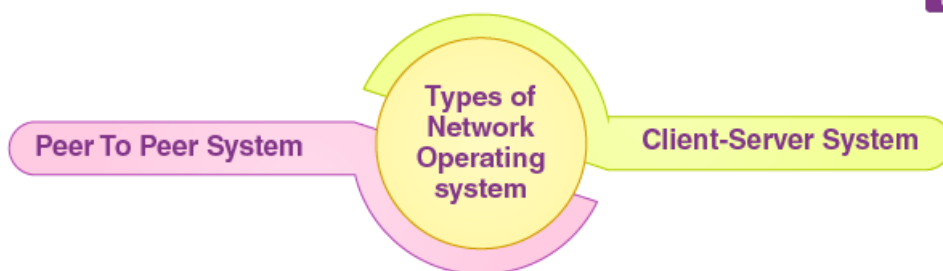
Cons of Multitasking OS

In a multitasking environment, numerous processors are busy at the same time to finish any task; therefore, the CPU generates more heat.

5. Network Operating System



A Network OS is a type of Operating System that incorporates software and protocols for communicating with other computers over a network in a convenient and cost-effective manner. Read more on [Network OS](#) here.



Pros of Network OS

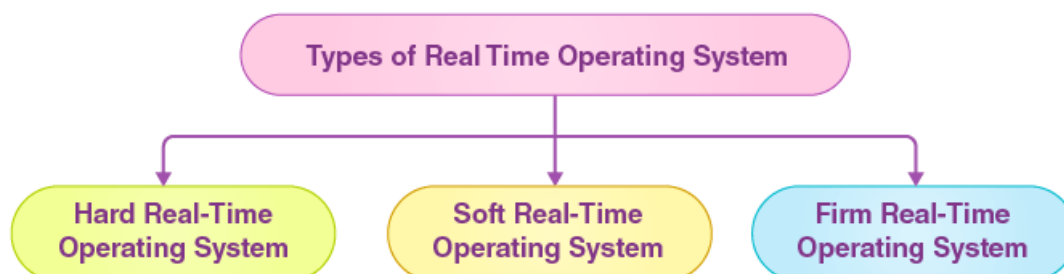
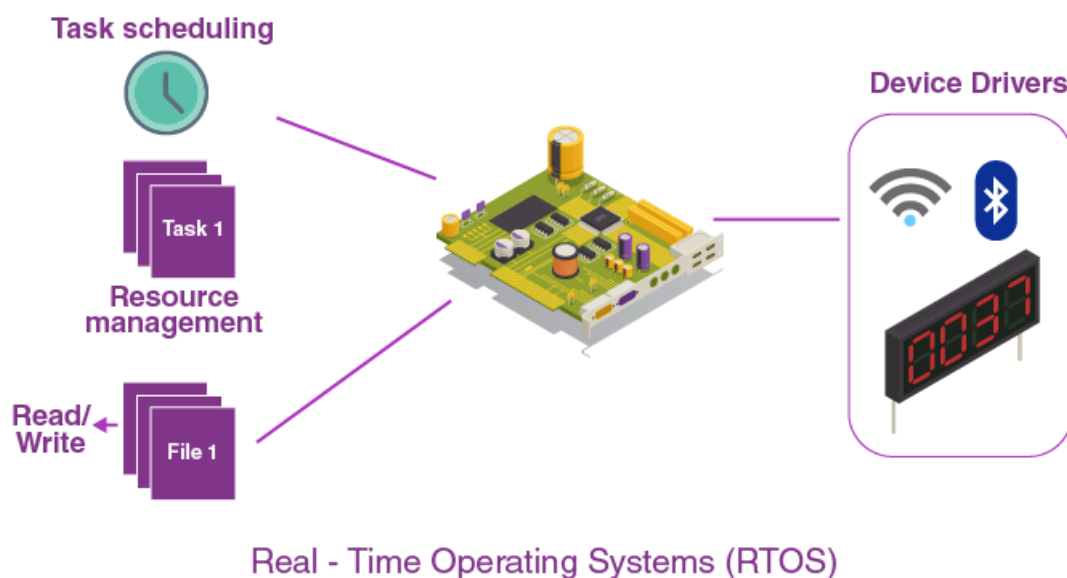
Because clients and servers are separated in this Operating System, network traffic is reduced. Setting up and maintaining this type of system is less expensive.

Cons of Network OS

The failure of a node in a system impacts the entire system in this form of the Operating System. Security and performance are critical considerations; as a result, network administration requires qualified network administrators.

6. Real-Time Operating System

In this type of system, each job has a deadline by which it must be completed; otherwise, there will be a significant loss, or even if the output is provided, it will be utterly useless. For example, in military applications, if you wish to drop a missile, the missile must be dropped with a specific degree of precision. Click [here](#) to read more on [Real-Time OS](#).



Pros of Real-Time OS

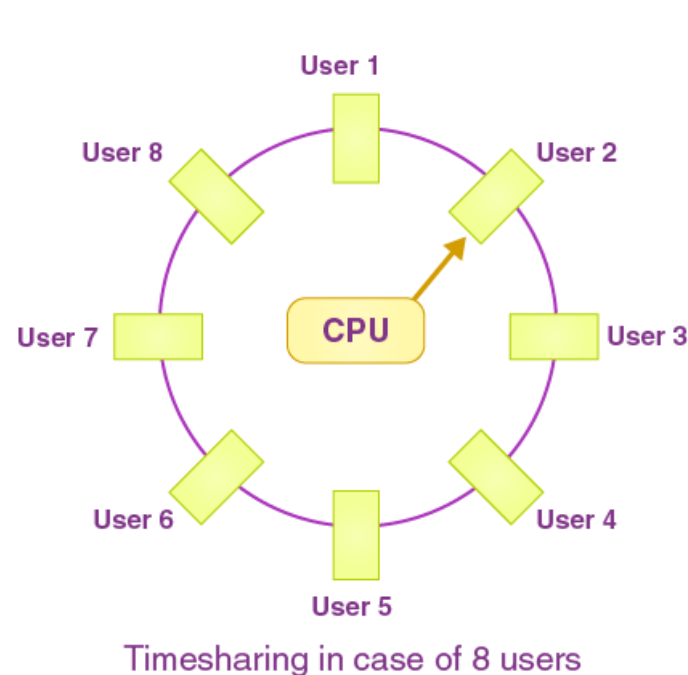
Under the real-time OS, it's simple to design, create, and run real-time applications. The maximum use of devices and systems is possible with a real-time Operating System.

Cons of Real-Time OS

The development of real-time Operating Systems is extremely expensive. Real-time Operating Systems are quite complex and can eat up a lot of CPU time.

7. Time-Sharing Operating System

The Time-Sharing OS provides computer resources to numerous programs at the same time in a time-dependent manner. As a result, it aids in providing direct access to the main computer to a large number of users. It's a natural progression from multiprogramming. The CPU is swapped between numerous programs provided by the different users in time-sharing on a scheduled basis.



Because a time-sharing OS allows multiple users to be served at the same time, it necessitates sophisticated CPU scheduling algorithms and input/output management. Building time-sharing Operating Systems are complex and expensive. Learn more about [Time-Sharing OS](#) here.

Pros of Time-Sharing OS

The time-sharing OS allows for efficient resource use and sharing. The CPU idle time and response time are reduced using this technology.

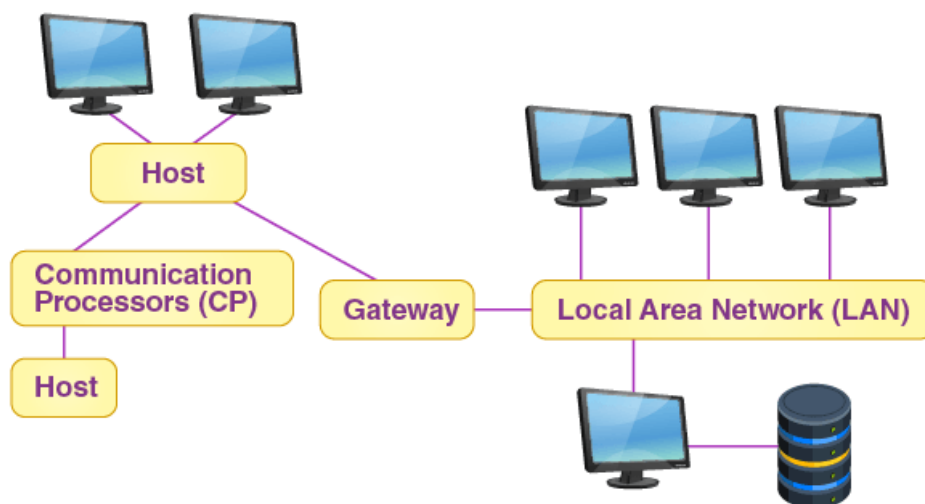
Cons of Time-Sharing OS

In comparison to other technologies, data transmission rates are extremely high.

As numerous users access a system at the same time, the security and integrity of user programs loaded in memory and data must be maintained.

8. Distributed Operating System

The Distributed OS is separated into sections and loaded on different machines rather than being placed on a single machine. Each machine has a piece of the distributed OS installed to allow them to communicate. Because they must deal with a variety of networking protocols, distributed Operating Systems are far more sophisticated, massive, and complex than network Operating Systems. Read more on [Distributed OS](#) here.



A typical view of a distributed System

Pros of Distributed OS

The distributed OS allows resources to be shared. This system is designed to be fault-tolerant.

5. Describe System Calls with suitable examples.

User programs cannot directly access hardware or critical OS resources because it would make the system unstable and insecure. To maintain safety, the operating system provides system calls — controlled interfaces that allow user programs to request services from the kernel. These calls act as a gateway between user mode and kernel mode. System Calls are,

- A way for programs to interact with the operating system.
- Provide the services of the operating system to the user programs.
- Only entry points into the kernel are executed in kernel mode.



System Call

Example:

- Opening a file in C (fopen) internally uses system calls like open().
- Running a program in Linux uses fork() and exec() system calls.
- Printing on screen uses the write() system call.

How do System Calls Work?

A system call is a controlled entry point that allows a user program to request a service from the operating system. Here's how it works:

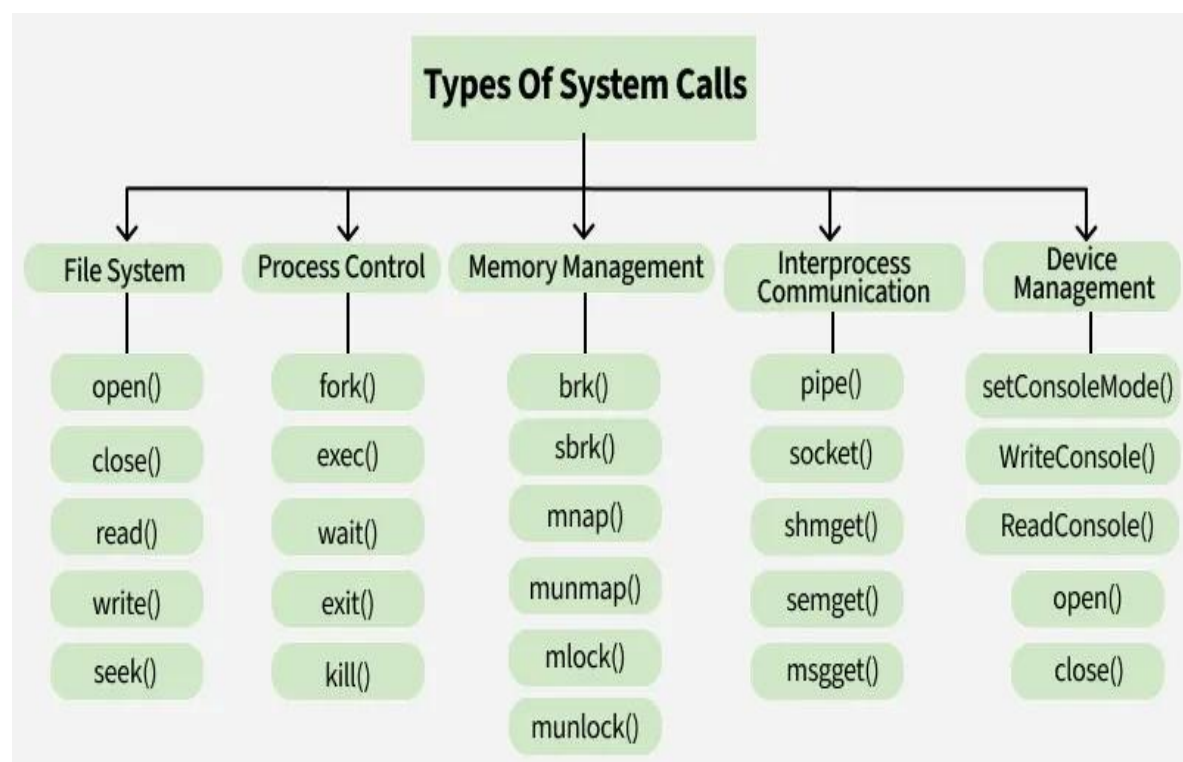
- The user program executes a system call instruction (e.g., using syscall or int 0x80).
- The CPU switches from user mode → kernel mode for safe execution.
- The kernel identifies the system call number and performs the requested operation (file access, process creation, memory allocation, etc.).

- After completing the task, the kernel switches back to user mode.
- The result (success/failure/data) is returned to the program.
- Without system calls, every program would need its own way to access hardware, leading to inconsistent and insecure systems.

System calls do not always cause context switching. They primarily involve a **mode switch** from user mode to kernel mode. A **context switch happens only when the calling process is blocked**, not during every system call.

Types of System Calls

Services provided by an OS are typically related to any kind of operation that a user program can perform like creation, termination, forking, moving, communication, etc. Similar types of operations are grouped into one single system call category. System calls are classified into the following categories:



Types of system call

- **File System:** Used to create, open, read, write, and manage files and directories.
- **Process Control:** Used to create, execute, synchronize, and terminate processes.
- **Memory Management:** Used to allocate, deallocate, and manage memory for processes.
- **Interprocess Communication (IPC):** Used for data exchange and communication between different processes.
- **Device Management:** Used to request and release devices, and to perform read/write operations on them.